

## Философская основа подхода

Рассматривается соревновательная деятельность спортсменов – единоборцев как скрытый процесс, эффективность которого мы не можем оценивать напрямую (например, стойки и маневрирование (24 значения), КФВ-29 значений, выведение из устойчивого положения -5 значений), мы можем наблюдать лишь последовательность оценок судей за *ЗАП*.

*Скрытая марковская модель* (Hidden Markov Model –HMM) позволяет по последовательности таких оценок определить наиболее вероятную последовательность скрытых состояний, то есть "диагностировать" причинно-следственную цепочку.

---

### Алгоритм решения обратной задачи

#### Этап 1: Подготовка и Определение Модели

**Цель:** Формализовать проблему в терминах HMM.

##### 1. Определение скрытых состояний {Sj, j=1..3}:

###### ○ S1: стойки и маневрирование (24 значения)

24 значения делятся на 2 группы маневрирование в правосторонней стойке и маневрирование в левосторонней стойке по 12 значений 0 и 1 в каждой группе. Целесообразно поставить в соответствие каждому эпизоду маневрирования числовое значение по формуле:  $\sum_{i=0}^{11} s1(i) * 2^i + 1$ , где s1(i) значение(0 или 1) в i ячейке группы (ячейки нумеруются слева направо от 0 до 11). Аналогично определяем числовые значения для эпизода маневрирование в левосторонней группе. Наборы значений представляются числами от 1. Далее логарифмируем эти числа по основанию 2 и при этом получаем вещественное число в диапазоне от 0 до 12. Эти данные представляют в удобной числовой форме результаты маневрирования спортсменов в каждом эпизоде схватки.

###### ○ S2: КФВ(j=29 значений)

Это событие разбито на 5 групп

захваты: их 15 видов, по аналогии с предыдущим разделом кодируем по формуле

$\sum_{i=0}^{14} s1(i) * 2^i + 1$  после логарифмирования получаем спектр значений от 0 до 14

хваты: 3 вида соответственно после аналогичных манипуляций с данными получаем спектр значений кодировки от 0 до 2

обхваты: аналогично – спектр значений в кодировке от 0 до 2

прихваты: 4 вида и спектр значений в кодировке от 0 до 3

упоры: 4 вида и спектр значений в кодировке от 0 до 3

###### ○ S3: выведение из устойчивого положения ВУП (5 значений), соответственно спектр значений после кодировки 0 до 4

###### ○ Временной интервал эпизода (время эпизода) характеризует динамику эпизода в модели играет роль дополнительной размерности, характеризующей мощность поединка.



- **Матрица эмиссии (В):**  $b_j(k) = P(\text{ЗАП в момент } t \mid \text{состояние } S3 \text{ в момент } t)$  – матрица переходов из  $s3(\text{ВУП})$  в ЗАП имеет вид для подмножества данных в легкой весовой категории:

|           | ВУП  | Броски | Удержания | Болевые |
|-----------|------|--------|-----------|---------|
| ВУП       | 0,62 | 0,19   | 0,00      | 0,19    |
| Броски    | 0,00 | 0,72   | 0,14      | 0,14    |
| Удержания | 0,00 | 0,00   | 1,00      | 0,00    |
| Болевые   | 0,00 | 0,00   | 0,00      | 1,00    |

Отражает, какие результаты ЗАП вероятны при сценарии атаки  $s1 \rightarrow s2 \rightarrow s3$  которая всегда завершается переходом из ВУП в состояние ЗАП.

- Следует так-же, учесть длительность каждого эпизода схватки  $\Delta t(i)$ , где  $i$  номер эпизода. В модели это фактор играет роль дополнительной размерности.
- **Начальное распределение вероятностей ( $\pi$ ):**  $\pi = P(\text{состояние ЗАП в момент } t=1)$ .
  - Вероятность того, с какого состояния система начала работу. В отсутствии данных можно задать равномерное распределение или считать  $\pi$  (Эффективная деятельность) близкой к 1.
- Для тестирования адекватности алгоритма преобразования исходных данных создадим и обучим нейронную сеть для предсказания сумм по столбцам в состоянии ЗАП на данных в столбцах состояний  $s1 \rightarrow s2 \rightarrow s3$ .

Преобразование данных описанное выше было протестировано на адекватность с использованием двухслойной нейронной сети модель которой создана в среде tensorflow на языке Python

```
import tensorflow as tf
import pandas as pd
from tensorflow import keras
from tensorflow.keras import layers

# 1. Загрузите данные из файла Excel в DataFrame pandas
# Для файлов .xlsx может потребоваться установить openpyxl: pip install openpyxl
file = 'c:/app/python_work/project/example.xlsx'
df = pd.read_excel(file,
                  sheet_name = 0,
                  )
dfy = pd.read_excel(file,
                  sheet_name = 1,
                  )
# sheet_name можно не указывать, если нужен первый лист

# 2. Преобразуйте DataFrame pandas в тензор TensorFlow
# Значения DataFrame автоматически конвертируются в тип tf.float32
tensor_from_excel = tf.convert_to_tensor(df.values)
tensor_from_excel_y = tf.convert_to_tensor(dfy.values)
```

```

# Проверка результата
print("Форма тензора:", tensor_from_excel.shape) # (количество_строк,
количество_столбцов)
print("Форма тензора:", tensor_from_excel_y.shape) # (количество_строк,
количество_столбцов y)
print("Тип данных:", tensor_from_excel.dtype)
print("Сам тензор:\n", tensor_from_excel)
print("Сам тензор:\n", tensor_from_excel_y)
# Создание новой последовательной модели
model_1 = keras.Sequential()

# Добавление слоев

model_1.add(layers.Dense(
    8, # Количество нейронов
    input_dim=8, # Задаем размер входа, 8
    activation='relu' # Используем функцию активации ReLU, так как все входы
положительные
))
# model_1.add(layers.Dropout(0.2,name='dropout'))

model_1.add(layers.Dense(
    1, # Количество нейронов. Мы хотим один выход
    activation='relu'
))
# Компиляция слоев в модель

model_1.compile(
    loss='mean_squared_error', # Функция потерь, которую мы минимизируем
    optimizer='adam', # Наша функция оптимизации
    metrics=['accuracy'] # Метрики - различные значения, которые вы хотите отслеживать во
время обучения
)
# Наша функция, которая принимает два числовых входа и возвращает соответствующий
булев результат

model_1.fit(
    tensor_from_excel, # Входные данные для обучения
    tensor_from_excel_y, # Выходные данные для обучения
    epochs=2000, # Количество итераций, на которое мы хотим обучить модель
    verbose=1 # Уровень детализации при выводе в терминале во время обучения
)

```

Модель состоит из 2 слоев –в первом слое 8 нейронов ,во втором -выходном слое 1 нейрон. На вход подавался тензор [80,8] преобразованных данных из результатов соревновательной деятельности в легчайшем весе, выход – тензор - столбец судейских баллов ( оценок эпизода).

Данные считывались из исходных данных файла EXCEL и преобразовывались в структуры тензоров tensorflow. Обучение сети осуществлялось на 2000 эпох(раз)

Результаты:

Epoch 1998/2000

3/3 [=====] - 0s 5ms/step - loss: 0.0020 - accuracy: 0.9241Epoch  
1999/2000

3/3 [=====] - 0s 6ms/step - loss: 0.0020 - accuracy: 0.9241Epoch  
2000/2000

3/3 [=====] - 0s 5ms/step - loss: 0.0020 - accuracy: 0.9241

с.к.о =0,2%, точность 92,4 % для предсказания результатов эпизода с помощью преобразованных данных.

Вывод – алгоритм преобразования данных адекватно представляет исходные данные модели.

Для полноты картины следует обучить сеть на полном наборе доступных данных по всем весовым категориям.

**Преобразованные данные позволяют сократить размер матриц модели НММ.**

## Этап 2: Обучение Модели (Обучение Баума-Велча)

**Цель:** Настроить параметры модели  $\lambda = (A, B, \pi)$  на основе исторических данных.

### 1. Сбор обучающей последовательности:

- Собрать данные за доступный длительный соревновательный период (например, по всем весовым категориям), где для каждого временного интервала известна последовательность наблюдаемых действий в цепочке состояний  $s_1 \rightarrow s_2 \rightarrow s_3$  и при этом известны итоговые результаты (состояния ЗАП).

### 2. Применение алгоритма Баума-Велча:

- Это алгоритм ЕМ-типа (Expectation-Maximization).
- **Е-шаг:** Используя текущие параметры  $\lambda$ , вычисляются ожидаемые частоты переходов между состояниями и эмиссии символов.
- **М-шаг:** Параметры  $\lambda$  пересчитываются так, чтобы максимизировать правдоподобие наблюдаемых данных.
- Алгоритм итеративно повторяет шаги Е и М до сходимости (когда изменение правдоподобия становится меньше заданного порога).

**Результат этапа:** Мы получаем обученную НММ, которая адекватно описывает историческое поведение нашей системы.

## Этап 3: Решение Обратной Задачи (Вывод Наиболее Вероятной Причины)

**Цель:** Для новой, текущей последовательности наблюдаемых событий (ЗАП) определить наиболее вероятную последовательность скрытых состояний (причин).

### 1. Формирование входной последовательности:

- Зафиксировать последовательность наблюдаемых символов за последние N эпизодов схваток.

## 2. Применение алгоритма Витерби:

- Это алгоритм динамического программирования, который находит **наиболее вероятную последовательность скрытых состояний** которая могла породить данную последовательность наблюдений ЗАП.
- Алгоритм вычисляет максимальную вероятность по пути в графе состояний, эффективно отсекая заведомо неоптимальные маршруты.

## 3. Интерпретация результата:

- Алгоритм выдаст последовательность, например:  $S = (S1, S1, S2, S2, S3)$ .
- **Диагноз:**
  - \*Периоды 1-2:\* Система находилась в состоянии S1 (маневрирование).
  - \*Периоды 3-4:\* Произошел переход в состояние S2 (КФВ). Это **первичная причина** формирования оценок ЗАП.
  - *Период 5:* Переход к состоянию S3 (ВУП), что привело к ЗАП **вторичная, производная причина**.

## Этап 4: Визуализация и Поддержка Принятия Решений

**Цель:** Предоставить результат в понятной для лица, принимающего решения, форме.

### 1. Построение "Диагностической траектории":

- Визуализировать последовательность состояний, найденную алгоритмом Витерби, на временной линии. Это наглядно показывает "историю" системы.

### 2. Расчет вероятностного профиля:

- Использовать **алгоритм прямого-обратного хода** (Forward-Backward) для расчета  $y_t(i)$  — вероятности находиться в состоянии  $S_i$  в момент  $t$  при условии всей последовательности наблюдений ЗАП.
- Это дает не просто одно "лучшее" предположение, а распределение вероятностей по всем состояниям для каждого момента времени. Это полезно, если наиболее вероятные гипотезы имеют близкие вероятности.

### 3. Формирование отчета с рекомендациями:

- Если диагностировано состояние S2 первичность КФВ, тогда нужно в методических рекомендациях обратить внимание на важность КФВ
- Если диагностировано состояние S3 (ВУП): Рекомендовать уделить внимание методикам отработки ВУП

## Блок-схема алгоритма (текстовое описание)

text

[Начало]

|

v

[Сбор исторических данных] -> [Обучение НММ (Алгоритм Баума-Велча)] -> [Сохранение обученной модели  $\lambda=(A,B,\pi)$ ]

|

|

v

v

[Формирование текущей последовательности симптомов O] --> [Применение алгоритма Витерби к O и  $\lambda$ ]

|

|

v

v

[Вывод и интерпретация] <-- [Визуализация диагностической траектории и вероятностного профиля]

|

v

[Формирование рекомендаций для ЛПР]

|

v

[Конец]

#### Преимущества подхода

- **Учет временной динамики:** Проблема рассматривается не как статичная, а как развивающийся во времени процесс.
- **Работа с неопределенностью:** Модель оперирует вероятностями, что адекватно отражает реальность, где причины часто неочевидны.
- **Объективность:** Решение основывается на данных и формальной модели, а не только на интуиции.
- **Прогнозирование:** Обученную модель можно использовать для прогнозирования будущих состояний тренировочных практик.

Этот алгоритм превращает задачу поиска причин из искусства в наукоемкую, Data-Driven процедуру, значительно повышающую обоснованность управленческих решений.